You said:

37_climber-main(2...
File

Lab_4_VibeCoding...
PDF

I'm working on my Software Engineering Lab 4 VibeCoding assignment. I have forked the Climber Repair Lab repository, which contains a Python Pygame vertical-platformer game in a single file called `game.py`.

Task 1: Fix the height-score bug.

Currently, the displayed height recalculates every frame and decreases when the player falls or moves down. The height should represent the maximum height reached during the current run and must never decrease.

Please inspect the existing code, identify the exact cause of the bug, and provide the minimal code change needed to fix it. Preserve the existing gameplay, scoring system, camera behavior, collision logic, and coding style. Explain what changed and why.

ChatGPT said:

Bro, I inspected the actual `game.py` from your uploaded Climber Repair Lab repository. The bug is in `Game.update()`, and the fix is just **one line**. You don't need to change the gameplay, camera, collision logic, or scoring system.

1. Exact cause of the bug

Your current code contains:

```
current_height = max(0, (HEIGHT - 40 - self.player.rect.y) // 10)
self.height = current_height
```

The problem is the second line:

```
self.height = current_height
```

Every frame, it **overwrites the previously recorded height** with the player's current height. When the player falls or moves down, the displayed height decreases.

2. Minimal code change

Replace those two lines with the following:

```
current_height = max(0, (HEIGHT - 40 - self.player.rect.y) // 10)
self.height = max(self.height, current_height)
```

That's it — **one line changed, with no other modifications.**

3. Why this fixes the bug

The `max()` function compares the highest height already recorded with the player's current height and keeps whichever is greater.

Situation	Previous height	Current height	Displayed height
Player climbs	10 m	15 m	15 m
Player falls	15 m	8 m	15 m
Player climbs higher	15 m	22 m	22 m
Player falls again	22 m	12 m	22 m

The height now represents the **maximum height reached during the current run**.

Your existing `self.reset()` method already initializes `self.height = 0`, so restarting the game correctly resets the height for the new run.

4. What remains unchanged

- **Gameplay:** Jumping and horizontal movement.
- **Scoring:** `score()` still combines height and coin score.
- **Camera:** Existing upward-following camera behavior.
- **Collision logic:** Platform collision and landing detection.
- **Lives and respawning:** Existing behavior is preserved.
- **Coding style:** Same variables and structure.

5. Prompt for your VibeCoding lab record

Since your Lab 4 requires you to document the prompts used to fix and enhance the repository, you can use this as your first prompt.

Writing

Prompt 1 — Fix the Height-Score Bug

Inspect the existing `game.py` file and identify why the displayed height decreases when the player falls or moves down. Modify only the code necessary to make the height represent the maximum height reached during the current run. Preserve the existing gameplay, scoring system, camera behavior, collision logic, and coding style. Ensure the height resets when a new run starts. Explain the cause of the bug and the minimal code change made.

Expected result: The height counter increases when the player reaches a new maximum height and never decreases during the same run. Restarting the game resets it to zero.

You can now apply this change to your forked repository and test it before moving on to the next task.

You said:

Continue working on my existing Climber Repair Lab game.py file.
Task 1, the height-score bug, has been addressed.
Task 2: Implement the function `platform_color(index, total)`.
The function is called during platform generation. The index represents a platform's position in the climb, where index 0 is the ground, and total is the total number of generated platforms.
Implement a smooth RGB colour transition so that lower platforms are green, middle platforms gradually become teal, and higher platforms become purple. Return a valid RGB tuple and handle edge cases safely.
Keep the implementation compatible with the existing Platform class.
Show the exact code to replace and explain how the colour progression works.

Worked for 11s

ChatGPT said:

For Task 2, I checked your uploaded `game.py`. The function `platform_color(index, total)` currently contains only `pass`, so it doesn't implement any colour transition yet.

The existing `Platform` class already calls this function correctly:

```
self.color = platform_color(index, total) or (100, 180, 100)
```

We'll replace only the function body, keeping the `Platform` class and platform-generation logic unchanged.

1. Exact code to replace

Find this code near the top of `game.py`:

```
def platform_color(index, total):
    """Return an (r, g, b) colour override for the platform at this index (0 is the ground), or None for the default green."""
    pass
```

Replace it with:

```
def platform_color(index, total):
    """Return an RGB colour based on the platform's height in the climb."""
    green = (70, 190, 90)
    teal = (30, 180, 170)
    purple = (150, 70, 200)

    if total <= 1:
        return green

    t = max(0.0, min(1.0, index / (total - 1)))

    if t <= 0.5:
        start = green
        end = teal
```

```
        t = t * 2
    else:
        start = teal
        end = purple
        t = (t - 0.5) * 2

    return tuple(
        int(start[i] + (end[i] - start[i]) * t)
        for i in range(3)
    )
```

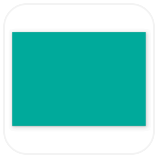
2. How the colour progression works



Lower platforms — Green

RGB: (70, 190, 90)

The ground and lower platforms begin green.



Middle platforms — Teal

RGB: (30, 180, 170)

Green gradually transitions into teal around the midpoint.



Higher platforms — Purple

RGB: (150, 70, 200)

Teal gradually transitions into purple toward the top.

The function calculates a normalized position `t` between `0.0` and `1.0`.

- `t = 0.0` represents the ground.
- `t = 0.5` represents the middle.
- `t = 1.0` represents the highest platform.

Each RGB channel is interpolated separately, producing intermediate colours instead of abrupt changes.

The function also handles edge cases:

- `total <= 1`: Returns green to avoid division by zero.
- **Negative index:** Clamps the position to the beginning of the gradient.

- **Index beyond the platform range:** Clamps the position to the end of the gradient.
- **Return type:** Always returns a three-integer RGB tuple.

3. Why this is compatible with your existing game

Your existing `Platform` class already stores the returned colour in `self.color`. No changes to `Platform`, `generate_platforms()`, drawing, movement, or collision detection are needed.

One important detail: your ground platform is created with index `0`, so it receives the green colour automatically. The existing fallback colour remains available if the function ever returns `None`, although this implementation always returns a valid RGB tuple.

4. VibeCoding lab prompt

Writing

Prompt 2 — Implement Platform Colour Progression

Inspect the existing `game.py` file and implement `platform_color(index, total)`. Use smooth RGB interpolation so lower platforms are green, middle platforms transition to teal, and higher platforms transition to purple. Handle edge cases safely, including a total of zero or one and indices outside the expected range. Return a valid RGB tuple and preserve compatibility with the existing Platform class. Do not modify gameplay, scoring, camera behavior, platform generation, or collision logic. Explain the implementation and the colour progression.

After applying the change: Run your existing game and check that the platforms progress from green at the bottom through teal in the middle to purple higher up.

You said:

Continue modifying my existing Climber Repair Lab game.py file. The height-score bug has been fixed, and progressive platform colours have been implemented.
Task 3: Implement `moving_platform_speed(index, total)`.
The existing Platform class already supports horizontal movement, bouncing between movement bounds, and carrying the player when

they stand on a moving platform.
Implement the function so that every third elevated platform moves horizontally at a small, controlled speed, while other platforms remain stationary. The ground platform must remain stationary. Use the existing movement mechanism instead of unnecessarily rewriting it. Provide the exact function implementation and any minimal changes needed to integrate it. Explain how to test the moving platforms in the game.

ChatGPT said:

Bro, Task 3 can be implemented with a small change. Your existing `Platform` class already handles horizontal movement, reversing direction at the bounds, and carrying the player. We only need to implement `moving_platform_speed(index, total)`.

1. Exact function implementation

In `game.py`, find:

```
def moving_platform_speed(index, total):
    """Return a horizontal oscillation speed in pixels/frame for the platform at this
index, or None/0 to keep it static."""
    pass
```

Replace it with:

```
def moving_platform_speed(index, total):
    """Return a horizontal oscillation speed in pixels/frame for the platform at this
index, or None/0 to keep it static."""
    if index <= 0 or index % 3 != 0:
        return 0

    return 1.5
```

This makes every third elevated platform move at a controlled speed of `1.5` pixels per frame. All other platforms remain stationary.

2. Why this works with your existing code

Your `Platform` constructor already contains:

```
self.speed = (moving_platform_speed(index, total) or 0) if movable else 0
```

And your `Game.update()` already calls:

```
for plat in self.platforms:
    dx = plat.update()
    if dx and self.player.standing_on is plat:
        self.player.rect.x += dx
```

So **you don't need to modify either of these sections**.

- `index <= 0` ensures the ground never moves.
- `index % 3 != 0` keeps non-third platforms stationary.
- `return 1.5` enables horizontal movement for every third elevated platform.
- `Platform.update()` reverses direction at the existing bounds.
- `Game.update()` carries the player when standing on a moving platform.

3. How to test it

1. Save `game.py` .
2. Open the terminal in your project directory.
3. Run:

   ```
   python game.py
   ```

4. Observe the platforms as the game runs:
   - Ground platform: stationary.
   - Platforms with indices 1 and 2: stationary.
   - Platform with index 3: moving horizontally.
   - Platforms with indices 4 and 5: stationary.
   - Platform with index 6: moving horizontally.
5. Jump onto a moving platform and check whether it carries you horizontally.
6. Watch it reach a movement boundary and reverse direction.

The platform indices are assigned during generation, so the moving pattern is determined by each platform's index, not its screen position.

4. VibeCoding lab prompt

Writing

Prompt 3 — Implement Moving Platforms

Inspect the existing `game.py` file and implement `moving_platform_speed(index, total)`. Make every third elevated platform move horizontally at a small, controlled speed while all other platforms remain stationary. The ground platform at index 0 must never move. Reuse the existing Platform class movement, boundary-bouncing, and player-carrying mechanisms. Make only the minimal changes necessary and explain how to test the result. Do not rewrite existing movement, collision, camera, or scoring logic.

Task 3 is complete once you've applied the function and verified the movement in-game. Your next step is to save the updated `game.py` and keep this prompt in your lab's chat-history deliverable.

You said:

Continue working on my existing Climber Repair Lab game.py file. The height-score bug has been fixed, platforms change colour as the player climbs, and every third elevated platform moves horizontally.
Task 4: Implement `on_coin_collected(coin, score)`.
The README specifies that this function is called immediately after a coin is collected, with the collected Coin object and the updated score. Its return value is ignored.
Implement a visible sparkle effect at the collected coin's position. The effect should be animated briefly and then disappear. Integrate it with the existing Game update and draw loop, using a lightweight particle or sparkle system without external image or sound assets.
Preserve the existing 50-point coin reward, coin removal, collision detection, and gameplay. Ensure the function signature matches `on_coin_collected(coin, score)` and that the effect is actually

visible when collecting a coin.
Show all necessary code changes and explain how to test the effect. If any existing code needs to be modified, identify the exact changes rather than replacing unrelated functionality.

Worked for 16s

ChatGPT said:

Bro, Task 4 needs one important integration change: implementing `on_coin_collected(coin, score)` alone won't make a sparkle visible because its return value is ignored. We need to create the particles when a coin is collected, update their animation every frame, and draw them using the existing camera offset.

I inspected the `game.py` inside your uploaded repository ZIP. The existing `Game.update()` already adds **50 points**, marks the coin as collected, calls `on_coin_collected(coin, self.score())`, and removes collected coins. We'll preserve that behavior and add a lightweight Pygame particle effect without external images or sound files.

1. Add the imports

At the top of `game.py`, add `math` to the existing imports:

```
import random
import math
import pygame
```

2. Replace `on_coin_collected()`

Find the existing function:

```
def on_coin_collected(coin, score):
    """Called the instant the player collects a coin, after its value has been added to the score. Add a sound or sparkle here."""
    pass
```

Replace it with:

```
def on_coin_collected(coin, score):
    """Create a brief sparkle effect at the collected coin's position."""
    coin.sparkles = []

    for i in range(8):
        angle = math.radians(i * 45)
        coin.sparkles.append({
            "x": float(coin.pos.x),
            "y": float(coin.pos.y),
            "vx": math.cos(angle) * 2.5,
            "vy": math.sin(angle) * 2.5,
            "life": 24
        })
```

This creates eight particles radiating outward from the coin's original position. Each particle has its own velocity and a lifetime of 24 frames.

3. Initialize the effect list in `Game.reset()`

Inside `Game.reset()`, find:

```
self.coins = spawn_coins(self.platforms)
```

Immediately below it, add:

```
self.coin_effects = []
```

This list keeps track of collected coins whose sparkle animations are still playing. The coins themselves will still be removed from `self.coins`.

4. Modify the coin-collection section in `Game.update()`

Find this existing section:

```
for coin in self.coins:
    if not coin.taken and self.player.rect.collidepoint(coin.pos):
        coin.taken = True
```

```
        self.coin_score += 50
        on_coin_collected(coin, self.score())
self.coins = [c for c in self.coins if not c.taken]
```

Replace it with:

```
for coin in self.coins:
    if not coin.taken and self.player.rect.collidepoint(coin.pos):
        coin.taken = True
        self.coin_score += 50
        on_coin_collected(coin, self.score())
        self.coin_effects.append(coin)

self.coins = [c for c in self.coins if not c.taken]

for coin in self.coin_effects:
    for particle in coin.sparkles:
        particle["x"] += particle["vx"]
        particle["y"] += particle["vy"]
        particle["vy"] += 0.08
        particle["life"] -= 1

    coin.sparkles = [
        p for p in coin.sparkles if p["life"] > 0
    ]

self.coin_effects = [
    c for c in self.coin_effects if c.sparkles
]
```

What changed?

- The collected coin is added to `self.coin_effects` so its particles can continue animating after the coin disappears.
- Each frame, the particles move outward and receive a small downward acceleration.
- Their lifetimes decrease until they expire.
- Empty effects are removed automatically, preventing particles from accumulating indefinitely.

5. Draw the sparkles in `Game.draw()`

Find this section:

```
for coin in self.coins:
    coin.draw(screen, self.cam_y)
self.player.draw(screen, self.cam_y)
```

Keep the existing coin and player drawing, and add the following block immediately after `self.player.draw(...)`:

```
for coin in self.coin_effects:
    for particle in coin.sparkles:
        x = int(particle["x"])
        y = int(particle["y"] - self.cam_y)
        size = max(1, int(4 * particle["life"] / 24))

        pygame.draw.line(
            screen, (255, 245, 170),
            (x - size, y), (x + size, y), 2
        )
        pygame.draw.line(
            screen, (255, 210, 70),
            (x, y - size), (x, y + size), 2
        )
```

The two crossing lines create a bright, star-like sparkle. The size shrinks as the particle's lifetime decreases, and `self.cam_y` ensures the animation stays aligned with the scrolling game world.

6. How to test Task 4

Step 1 — Run the game

Open the project folder in your terminal and execute:

```
python game.py
```

Step 2 — Collect a coin

Move and jump into a visible coin. You should see eight small golden sparkles radiating outward from its position.

Step 3 — Verify the behavior

- The coin disappears immediately.
- The coin score increases by 50.
- Sparkles animate briefly and disappear.
- The effect follows the camera correctly.
- You can continue playing normally while the effect animates.

7. Your VibeCoding prompt for Task 4

Writing

Prompt 4 — Implement Coin Collection Sparkle Effects

Inspect the existing game.py file and implement on_coin_collected(coin, score). When a coin is collected, create a visible animated sparkle effect at its position using lightweight Pygame particles. Integrate particle updates into Game.update() and rendering into Game.draw(), accounting for the camera offset. The particles must animate briefly and disappear without external assets. Preserve the existing 50-point reward, coin removal, collision detection, gameplay, and the exact function signature. Make only the necessary changes and explain how to test the effect.

Final check: Keep your Task 1, 2, and 3 implementations in your local file; the ZIP I inspected contains the original starter versions of those functions. The changes above are specifically for Task 4 and are designed to integrate with the existing `Game` loop. After testing all four tasks together, save the final `game.py` and record your after-changes gameplay video.